OSS 설계

개념

SW design(설계)

- 사용자의 요구사항을 충족시키기 위해 SW의 메서드, 함수, 객체, 코드의 전반적 구조와 상호작용을 정의하는 과정
- 이 과정은 SW 개발의 핵심 요소인 SW 프로세스를 이해하고, 각 단계의 필수 작업과 결과물을 학습하는 것이 목표, 특히 Unified Modeling Language를 활용한 OSS의 “SW 설계 및 분석” 단계에 중점

과정

1. Requirement Analysis (요구 사항 분석)
2. Design (설계)
    - Functional
    - Structure
    - Behavior
    - DB
    - UI/UX
3. Implementation (구현)
4. Testing (디버깅, 버그 픽스 등)
5. Maintenance (유지 보수)

OSS 프로젝트

요구 사항 분석

요구사항 수집

1. 헬린이 (운동을 시작하려는 사람)
   고민: “뭐부터 해야 할지 모름, 헬스장 머신 사용법도 모름”
    - 요구사항: 사용자 수준(Level)과 사용 가능한 운동 기구(맨몸, 덤벨, 바벨, 머신)에 따른 시작 가이드 제공
2. 운동에 동기부여가 필요
   고민: 매일 똑같은 루틴, 몸에 변화가 없어서 매너리즘에 빠짐
    - 요구사항: 시각적인 성장 그래프와 지난 달 대비 볼륨 증가량 알림 기능, 특정 목표 달성 시 ‘배지’ 부여, 운동 횟수에 따른 리그전
3. 루틴을 스스로 짜는 것에 어려움
   고민: 무슨 종목을 어떤 순서로 해야 효율적일까

- 요구사항: 타겟 부위(가슴, 등, 하체 등)만 정하면 검증된 루틴 템플릿 혹은 AI API를 활용하여 루틴을 불러오거나 생성하고, 루틴을 설명, 자신에 맞게 종목을 교체

4. 헬스 독학
   고민: 운동을 하는데 부정확한 자세로 인해 자극 감소와 통증 호소
   - 요구 사항: 운동 종목별 핵심 주의 사항과 관련 유튜브 영상 링크 혹은 운동 방법 제공
5. 다이어트를 시도하는 여성
   고민: 몸무게는 그대로인데 눈바디가 달라짐, 생리 주기로 인한 식욕 조절의 어려움
   - 요구사항: 단순 체중이 아닌 골격근, 체지방률 추이 시각화(인바디)
     - 신체 상태 기록: 컨디션, 부종 여부, 생리 주기 등 컨디션
     - 목표 설정: 체지방 -3kg 등 구체적 수치 목표와 달성률 표시
6. 식단 고민러
   고민: 매일 먹는 식단에 질림, 탄단지 비율이 적당한가
   - 요구사항: 음식명과 양을 입력하여 자동으로 영양성분 환산
   - 대체 식품 추천: 닭가슴살 대신 먹을 수 있는 단백질원 제안
   - 일일 권장량 가이드: 기초대사량 기반 권장 섭취량 대비 현재 섭취량 표시

페르소나	문제/상황(Why)	핵심 요구사항(What)	주요 입력 데이터	주요 출력/화면(Deliverable)	우선순위
입문자(헬린이)	무엇부터 시작할지 모르고, 기구 사용법이 낯섦	레벨/기구 기반 시작 가이드 + 쉬운 루틴 추천	운동 수준, 사용 기구(맨몸/덤벨/바벨/머신), 목표	온보딩 설문, 추천 루틴 카드, 운동별 기초 가이드	Must
정체기(동기부여 필요)	변화가 안 보여 매너리즘, 루틴이 지루함	성장 시각화 + 월간 비교 리포트 + 보상(배지/리그)	운동 기록(세트/횟수/무게), 기간	주간/월간 리포트, 성장 그래프, 배지/리그 화면	Should
루틴 미아(설계 어려움)	무슨 종목을 어떤 순서로 해야 할지 모르겠음	부위/목적 기반 표준 루틴 템플릿 + 커스텀(교체/순서 변경)	타겟 부위, 목적(근비대/다이어트/체력), 가용 시간	템플릿 목록, 루틴 편집 화면, 종목 교체 추천	Must
자세 고민(독학)	자세가 부정확해 자극이 줄고 통증이 생김	종목별 핵심 포인트/주의사항 + 영상/방법 링크 제공	선택 종목, 통증 부위(선택)	운동 디테일 페이지(포인트/주의/영상), 체크리스트	Must
다이어터(눈바디/체성분)	체중 변화보다 체형 변화를 보고 싶음	인바디/체성분 추이 + 눈바디 비교 + 목표/달성률	인바디(골격근/체지방률 등), 사진(선택), 목표 수치	체성분 그래프, 사진 비교(전/후), 목표 대시보드	Should
컨디션 관리(주기/부종 등)	컨디션에 따라 수행 능력이 달라짐	컨디션/부종/생리 주기 기록 + 기록 기반 인사이트	컨디션, 부종 여부, 생리 주기(선택)	컨디션 캘린더/타임라인, 요약 인사이트 카드	Could
식단 고민러(탄단지)	식단이 반복되고, 비율이 적정한지 모르겠음	음식 입력 → 영양성분 자동 환산 + 권장량 대비 현황	음식명, 섭취량, 목표(다이어트/유지/증량)	탄단지/칼로리 대시보드, 일일 권장량 게이지	Should
메뉴 다변화(대체식)	같은 식단이 질리고, 대체재를 잘 모름	유사 영양 성분 기반 대체 식품 추천	원식품(예: 닭가슴살), 선호/알레르기(선택)	대체 식품 리스트, 영양 비교 카드	Could

페르소나	문제/상황(Why)	핵심 요구사항 (What)	주요 입력 데이터	주요 출력/화면 (Deliverable)	우선순위
복귀 유저(재시작)	쉬었다가 다시 시작하려니 강도 설정이 어려움	최근 기록 기반 복귀 루틴 + 강도 자동 조정(완만한 재적응)	마지막 운동일, 최근 수행량, 목표	복귀 플로우, 추천 강도/볼륨, 첫 주 루틴	Should
시간 부족(10~20분)	시간이 없어서 루틴을 다 못 함	짧은 루틴(핵심 종목 위주) + 타이머/휴식 자동 관리	가능 시간, 기구, 타겟 부위	퀵 루틴 추천, 타이머/세트 진행 화면	Could
부상/통증 사용자	특정 동작에서 통증이 있고 악화가 걱정됨	통증 체크 + 대체 운동 추천 + 중단/주의 가이드	통증 부위/강도, 피해야 할 동작(선택)	안전 경고, 대체 운동 리스트, 금지/주의 표시	Should
기록 귀찮(최소 입력)	기록이 번거로워서 앱을 안 쓰게 됨	원탭 기록 + 자동 완성(최근 세트/무게) + 빠른 시작	최근 루틴/기록, 즐겨찾기 종목	빠른 기록 UI, 최근 기록 불러오기, 즐겨찾기	Must

요구사항 분석

1. 기능적 요구사항 (Functional Requirements)

시스템이 **무엇(What)**을 해야 하는지에 대한 구체적인 기능들

[운동 및 루틴 관리]

- **온보딩 및 맞춤 설정:** 사용자의 운동 수준(입문/중급/고급), 보유 기구, 목표에 따른 초기 설문 및 프로필 설정 기능.
- **루틴 템플릿 제공:** 타겟 부위와 목적(근비대/다이어트/체력)에 맞는 표준 루틴 불러오기 기능.
- **커스텀 루틴 편집:** 운동 종목의 순서 변경, 종목 교체 및 삭제 기능.
- **운동 가이드 제공:** 종목별 핵심 포인트, 주의사항(자세), 관련 영상 링크 및 체크리스트 출력 기능.

[기록 및 분석]

- **기록 입력:** 운동 세트별 무게, 횟수, 시간 및 신체 컨디션(부종, 생리 주기 등) 입력 기능.
- **성장 시각화:** 주간/월간 운동 볼륨(무게x횟수) 증가량 그래프 및 리포트 생성 기능.
- **체성분 관리:** 인바디 데이터(골격근량, 체지방) 기록 및 눈바디 사진 비교 기능.
- **식단 및 영양 관리:** 음식별 탄단지 자동 계산, 권장 섭취량 대비 현재 섭취량 가이드 및 대체 식품 추천 기능.

[동기부여]

- **보상 시스템:** 특정 목표 달성 시 배지 부여 및 사용자 간 리그(순위) 표시 기능.

---

2. 비기능적 요구사항 (Non-functional Requirements)

시스템의 **품질, 성능, 제약사항** 등 운영적인 측면에 대한 요구사항이야.

[객체지향 설계 및 확장성 - OSS 핵심]

- **확장성 (Extensibility):** 새로운 운동 종목, 루틴 템플릿, 또는 식단 알고리즘을 추가할 때 기존 코드를 수정하지 않고 확장 가능해야 함 (인터페이스 기반 설계).
- **유연성 (Flexibility):** 다양한 사용자 목표(다이어트, 증량, 유지)에 따라 탄단지 계산 방식이나 추천 로직이 동적으로 변경되어야 함 (전략 패턴 활용).

[사용성 및 효율성]

- **직관적인 인터페이스:** 입문자도 쉽게 따라 할 수 있는 온보딩 흐름과 운동 디테일 페이지의 가독성 확보.
- **데이터 지속성 (Persistence):** 로컬 환경(Java 앱)에서 종료 후에도 모든 운동 기록과 사용자 데이터가 안전하게 저장되고 불러와져야 함 (파일 I/O 또는 DB 연동).

[신뢰성 및 유지보수]

- **정확성:** 영양 성분 계산 및 운동 볼륨 합산 시 데이터 오류가 없어야 함.
- **가독성 및 문서화:** 오픈소스 프로젝트로서 다른 개발자가 구조를 이해할 수 있도록 Clean Code 원칙을 준수하고 UML 다이어그램이 포함된 문서가 제공되어야 함.

ID	요구사항명	유형	관련 페르소나	우선순위
F-01	레벨별 온보딩 설문	기능	입문자	Must
F-02	운동별 가이드 출력	기능	자세 고민	Must
F-03	탄단지 자동 계산	기능	식단 고민	Should
N-01	추천 로직 확장성	비기능	루틴 미아/식단 고민	High
N-02	데이터 로컬 저장	비기능	공통	High

설계

UML 다이어그램 (Mermaid)

```
classDiagram
    %% 1. Domain & Entities
    class User {
        -String name
        -double weight
        -double targetWeight
        -GoalType goal
        +updateProfile()
    }

    class Exercise {
        <<abstract>>
        -String name
        -String targetMuscle
        -String description
        +displayGuide()*
    }
```

```
class WeightExercise {
    -int sets
    -int reps
    -double weight
    +displayGuide()
}

class CardioExercise {
    -int durationMinutes
    -int intensity
    +displayGuide()
}

class Routine {
    -String routineName
    -List~Exercise~ exercises
    +addExercise(Exercise e)
    +getTotalVolume()
}

%% 2. Services & Logic
class WorkoutManager {
    -Routine currentRoutine
    -RecommendationStrategy recStrategy
    +setStrategy(RecommendationStrategy s)
    +recommendRoutine()
    +logWorkout()
}

class RecommendationStrategy {
    <<interface>>
    +recommend(User user)*
}

class BeginnerStrategy {
    +recommend(User user)
}

%% 3. Infrastructure (Data)
class Repository {
    <<interface>>
    +save(Object data)*
    +load() Object*
}
```

```
    class FileRepository {
        -String filePath
        +save(Object data)
        +load() Object
    }

    %% 4. Presentation (UI)
    class MainApp {
        +main(String[] args)
        +start(Stage stage)
    }

    %% Relationships
    Exercise <|-- WeightExercise : Inheritance
    Exercise <|-- CardioExercise : Inheritance
    Routine o-- Exercise : Aggregation
    WorkoutManager --> Routine : Manages
    WorkoutManager --> RecommendationStrategy : Strategy Pattern
    RecommendationStrategy <|.. BeginnerStrategy : Implementation
    WorkoutManager ..> Repository : Dependency Injection
    Repository <|.. FileRepository : Interface Realization
```

기능

1) 기능 범위(모듈) 정의

- **온보딩/프로필**
  - 운동 수준(입문/중급/고급), 목표(근비대/다이어트/체력), 가용 시간, 보유 기구, 통증/금기 동작(선택), 알레르기/기피 식품(선택) 입력
  - 결과: 기본 추천 전략 선택 + 초기 루틴/가이드 제시
- **루틴 관리**
  - 루틴 템플릿 탐색(부위/목적/시간/장비 필터)
  - 루틴 생성(AI 또는 템플릿 기반)
  - 루틴 편집(순서 변경, 종목 교체/삭제, 세트·반복·휴식 수정)
  - 즐겨찾기/최근 루틴/빠른 시작
- **운동 수행(세션)**
  - 세트 진행(타이머/휴식 자동)
  - 원탭 기록(최근 세트 자동완성, +1/-1 스텝)
  - 운동 가이드(주의사항, 체크리스트, 영상 링크)
  - 통증/불편감 체크(중단/대체 운동 추천)

- **기록/리포트**
  - 운동 볼륨(무게×횟수×세트) 집계, PR(최고 중량/최고 볼륨) 표시
  - 주간/월간 비교 리포트, 성장 그래프
  - 스트릭(연속 운동일) 및 목표 달성률
- **체성분/컨디션**
  - 인바디(골격근/체지방/체지방률 등) 기록
  - 눈바디 사진 전/후(선택) + 기간 비교
  - 컨디션(피로/수면/부종/생리 주기 등) 기록과 요약 인사이트
- **식단/영양(선택 범위)**
  - 음식 입력 → 영양 성분(칼로리/탄단지) 환산
  - 일일 권장량 대비 게이지, 대체 식품 추천
- **동기부여(게이미피케이션)**
  - 배지(마일스톤), XP/레벨, 리그(주간 랭킹), 데일리 퀘스트

2) 핵심 유스케이스(Use Case) 요약

- UC-01: 사용자가 온보딩을 완료하고 초기 루틴을 추천받는다.
- UC-02: 사용자가 '오늘 운동 시작'을 눌러 세션을 진행하고 세트 기록을 남긴다.
- UC-03: 사용자가 특정 종목의 가이드/영상/주의사항을 확인한다.
- UC-04: 사용자가 지난 4주 리포트에서 볼륨 증가와 PR을 확인한다.
- UC-05: 사용자가 루틴을 편집하고 즐겨찾기에 저장한다.
- UC-06: (선택) 사용자가 식단을 기록하고 탄단지/권장량 대비를 확인한다.

구조

1) 아키텍처(권장: Clean Architecture + 전략 패턴)

- **Presentation(UI)**
  - 화면/컴포넌트, 상태(뷰 모델), 사용자 입력 검증
  - 예: OnboardingView, RoutineView, WorkoutSessionView, ReportsView
- **Application(Use Case Layer)**
  - 유스케이스(서비스)들이 도메인 규칙을 호출하고 트랜잭션 단위를 구성
  - 예: RecommendRoutineUseCase, StartWorkoutSessionUseCase, LogSetUseCase, GenerateReportUseCase
- **Domain(Model)**
  - 엔티티/값 객체/도메인 서비스

    - 예: UserProfile, Routine, Exercise, SetRecord, BodyMetric, NutritionEntry
- **Infrastructure(Data)**
    - Repository 구현(File/DB/API), 외부 API(영양 DB, 영상 링크, AI API)

2) 주요 패키지/모듈 예시

- `domain/`
    - `entities/` (UserProfile, Routine, Exercise 등)
    - `services/` (VolumeCalculator, PRDetector 등)
    - `strategy/` (RoutineRecommendationStrategy, NutritionStrategy 등)
- `application/`
    - `usecases/` (RecommendRoutine, LogWorkout, GenerateReport)
    - `dto/` (요청/응답 모델)
- `infrastructure/`
    - `persistence/` (FileRepository, SqlRepository)
    - `external/` (AIClient, NutritionApiClient)
- `presentation/`
    - `view/`, `viewmodel/`, `components/`

3) 확장 포인트(OSS 관점)

- **추천 로직**: `RecommendationStrategy`를 추가 구현하여 새로운 추천 정책을 플러그인처럼 추가
- **운동 종목 타입**: Exercise 하위 클래스로 확장(머신, 케틀벨, 스트레칭 등)
- **저장소**: Repository 인터페이스로 파일 저장 ↔ DB 저장 전환 가능

행동

1) 주요 시퀀스(텍스트 시퀀스 다이어그램)

A. 온보딩 → 초기 루틴 추천

1. UI: 사용자 입력(레벨/장비/목표/시간)
2. UseCase: `RecommendRoutineUseCase` 호출
3. Domain: `RecommendationStrategy.recommend(profile)`
4. Data: 템플릿 조회 + (선택) AI 호출
5. 결과: 추천 루틴 + 이유(설명) + 주의사항 반환

B. 운동 세션 진행(원탭 기록)

1. UI: '운동 시작'

2. UseCase: `StartWorkoutSessionUseCase` → `WorkoutSession` 생성
3. UI: 세트 완료 버튼 → `LogSetUseCase`
4. Domain: 볼륨/PR 계산, 피로도/컨디션 반영(선택)
5. Data: `WorkoutSessionRepository.save(session)`
6. UI: 세션 종료 → 요약 카드(총 볼륨, PR, 부위별 볼륨)

C. 월간 리포트 생성

1. UI: 기간 선택(최근 4주)
2. UseCase: `GenerateReportUseCase`
3. Data: 기간 내 세션/세트 기록 조회
4. Domain: 볼륨 집계, 지난달 대비 증감, streak 계산
5. UI: 그래프/인사이트 카드 렌더

2) 도메인 규칙(예시)

- 볼륨 계산: $volume = \sum(weight \times reps)$ (체중/머신 등은 변형 가능)
- PR 정의(예):
  - 최고 중량 PR: 같은 종목에서 `max(weight)`
  - 최고 볼륨 PR: 같은 종목에서 `max(weight×reps)`
- 점진적 과부하(예): 최근 2회 평균 수행이 목표 반복을 넘으면 다음 세션에서 +2.5kg 또는 +1~2 reps 제안

DB

> 아래는 **RDB(PostgreSQL) 기준 스키마 초안**이야. (로컬 파일 저장으로 시작해도 동일한 논리 구조를 유지하면 DB로 확장하기 쉬움)

1) 핵심 엔티티(테이블) 설계

- **users**
  - `id (PK)`, `email (UNIQUE)`, `name`, `created_at`
- **user_profiles**
  - `user_id (PK/FK)`, `level`, `goal`, `available_time_min`, `equipment (jsonb)`, `injury_notes`, `preferences (jsonb)`
- **exercises**
  - `id (PK)`, `name`, `target_muscle`, `type` (weight/cardio/mobility), `equipment`, `difficulty`, `guide_text`, `video_url`
  - 검색 인덱스: `name`, `target_muscle`
- **routine_templates**

    - `id (PK)`, `title`, `goal`, `target_muscle`, `min_time`, `equipment`, `level`
- **routine_template_items**
    - `id (PK)`, `template_id (FK)`, `exercise_id (FK)`, `order_no`, `sets`, `reps`, `rest_sec`, `notes`
- **routines** (유저 커스텀 루틴)
    - `id (PK)`, `user_id (FK)`, `title`, `source` (template/ai/manual), `created_at`, `updated_at`
- **routine_items**
    - `id (PK)`, `routine_id (FK)`, `exercise_id (FK)`, `order_no`, `sets`, `reps`, `rest_sec`, `target_weight`, `notes`
- **workout_sessions**
    - `id (PK)`, `user_id (FK)`, `routine_id (FK, NULL 허용)`, `started_at`, `ended_at`, `mood`, `pain_level`, `memo`
    - 인덱스: `(user_id, started_at)`
- **set_records**
    - `id (PK)`, `session_id (FK)`, `exercise_id (FK)`, `set_no`, `weight`, `reps`, `duration_sec`, `rpe` (선택), `created_at`
    - 인덱스: `(exercise_id, created_at)`, `(session_id)`
- **body_metrics**
    - `id (PK)`, `user_id (FK)`, `measured_at`, `weight`, `skeletal_muscle`, `body_fat_mass`, `body_fat_pct`, `notes`
    - 인덱스: `(user_id, measured_at)`
- **photos** (선택)
    - `id (PK)`, `user_id (FK)`, `taken_at`, `type` (front/side/back), `file_url`, `notes`
- **nutrition_entries** (선택)
    - `id (PK)`, `user_id (FK)`, `ate_at`, `food_name`, `amount`, `kcal`, `protein_g`, `carb_g`, `fat_g`, `source`
    - 인덱스: `(user_id, ate_at)`
- **achievements / user_achievements**
    - 배지 마스터 테이블 + 유저 획득 기록
- **leagues / league_members / league_scores**
    - 주간 리그 계산을 위한 집계 테이블(또는 뷰)

2) 관계(요약)

- `users 1:1 user_profiles`
- `users 1:N routines`
- `routines 1:N routine_items` (각 item은 `exercise` 참조)

- `users 1:N workout_sessions`
- `workout_sessions 1:N set_records`
- `users 1:N body_metrics`, `users 1:N nutrition_entries`

3) 저장/조회 성능 포인트

- 최신 기록 자동완성을 위해 `set_records (exercise_id, created_at desc)` 인덱스가 중요
- 리포트는 기간 필터가 많으므로 `workout_sessions (user_id, started_at)` 인덱스
- 영양/체성분도 기간 그래프용으로 `(user_id, measured_at/ate_at)` 인덱스

UI/UX

1) 정보 구조(IA) / 주요 화면

- **Home/Dashboard**
  - 오늘의 퀘스트, 빠른 시작(최근 루틴), 최근 7일 스트릭, 핵심 그래프(볼륨/체성분)
- **Onboarding**
  - 3~5단계 스텝 퍼널(레벨→목표→장비→시간→(선택)통증/선호)
- **Routine**
  - 템플릿 목록(필터/검색) + 루틴 편집(드래그 정렬, 종목 교체)
- **Workout Session**
  - 세트 진행 카드(현재 종목, 세트 번호, 목표 reps/weight)
  - 큰 CTA: '세트 완료' / '휴식 시작'
  - 자동완성: 최근 기록을 기본값으로 프리필
- **Exercise Detail**
  - 핵심 포인트 3개, 주의사항, 체크리스트, 영상 링크
- **Reports**
  - 주간/월간 탭, 지난달 대비 증감, PR 카드, 부위별 볼륨 차트
- **Body/Nutrition(선택)**
  - 체성분 그래프 + 사진 비교
  - 탄단지/칼로리 게이지 + 식단 히스토리
- **Rewards/League**
  - 배지 컬렉션, 리그 랭킹(주간)

2) UX 원칙(입문자/기록 귀찮 페르소나 대응)

- **최소 입력**: 세트 기록은 기본값(최근 기록) 제공 + +/- 스텝퍼로 조정

- **즉시 피드백**: 세트 완료 후 '이번 세트 볼륨'과 '지난번 대비 +X%' 같은 1줄 피드백
- **진행 유도**: 세션 화면은 '다음 행동'이 항상 1개로 명확(완료/휴식/종료)
- **실수 방지**: 중량/횟수 비정상 값 입력 시 경고(예: 지난 기록 대비 3배 이상)
- **안전**: 통증 체크가 켜져 있으면 대체 운동과 중단 가이드가 우선 노출

3) 컴포넌트(재사용 단위)

- RoutineCard, ExerciseChip(부위/장비 배지), SetLogger(스텝퍼), RestTimer, PRBadge, InsightCard, ChartContainer

4) 접근성/가독성

- 큰 버튼/큰 숫자(세션 화면), 대비 높은 색, 진행 상태(현재 세트/총 세트) 명확 표시
- 초보자 용어 설명 툴팁(볼륨, PR, RPE 등)

5) MVP 컷(우선순위)

- **MVP(Must)**: 온보딩, 루틴 템플릿/편집, 세션 기록(원탭), 운동 가이드, 기본 리포트(주/월 볼륨)
- **V2(Should)**: 배지/리그, 인바디/사진, 식단/영양
- **V3(Could)**: 컨디션 인사이트 고도화, 부상/대체 운동 추천 정교화, AI 코칭 대화형 플로우

구현

디버깅

유지보수